

WORKMY

Apresentação da Arquitetura de Software

Uma análise técnica detalhada sobre a engenharia de software da plataforma, cobrindo desacoplamento de camadas, cache inteligente, persistência transacional e faturamento idempotente sob demanda.

Autor: Rafael Lass, Victor H. e Eric A.

Padrões de Projeto, Caching Local, Supabase Cloud & Django Ninja Engine

1. Visão Geral da Arquitetura (System Overview)

Estrutura Física e Lógica de 3 Camadas Distribuídas

- Sistemas Distribuídos Desacoplados

O ecossistema é fracionado em três tiers funcionais e físicos isolados: Apresentação, Aplicação REST e Persistência de Dados em Nuvem, promovendo escalabilidade independente.

- Desacoplamento Estrito de Views

Zero acoplamento de renderização no servidor. O cliente React executa inteiramente no browser e consome as APIs de forma assíncrona por meio de requisições AJAX.

- Comunicação HTTP Stateless

Cada requisição trafega de forma independente carregando a identidade do usuário criptografada no JWT, retornando dados estruturados em JSON.

Fundamentos e Roteamento

A arquitetura do WorkMy foi projetada aplicando diretamente os conceitos consolidados de sistemas multicamadas distribuídos:

1. O ciclo clássico de requisição-resposta HTTP é traduzido nos decoradores de rota tipados em Python.
2. O API Gateway gerencia permissões e valida payloads antes de atingir o banco de dados.
3. Toda transação de dados ocorre de forma atômica no Supabase, garantindo conformidade ACID.

Veja o diagrama detalhado da arquitetura geral na próxima página.

LOCALIZAÇÃO DO CÓDIGO FONTE (CAMINHOS RELATIVOS):

frontend/src/

-> Interface do Usuário (React.js SPA)

backend/api/

-> API Gateway e Rotas Rest (Django Ninja)

backend/core/settings.py

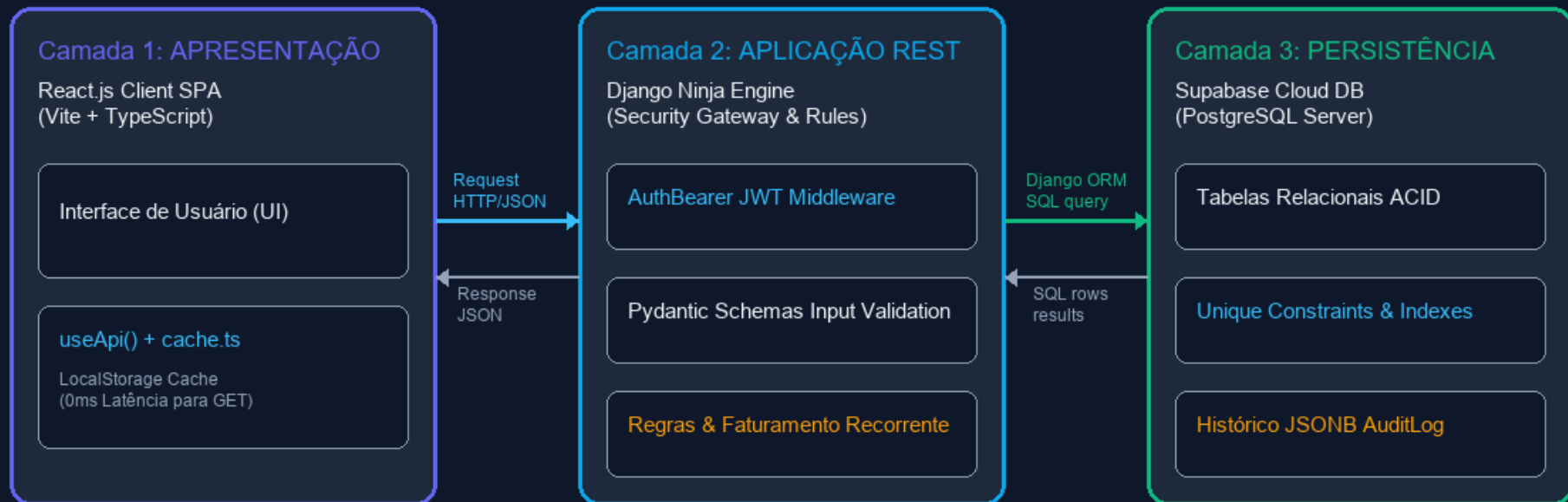
-> Parâmetros e drivers do Supabase Postgres

Visão Geral - Diagrama de Camadas da Arquitetura Geral

Mapeamento Físico de Camadas e Fluxo de Dados

Diagrama 1: Arquitetura Geral Multicamadas (Multi-Tier REST)

Fluxo de Comunicação Desacoplado: Client-Server Tier



2. Camada de Apresentação & Caching Estratégico

Mecanismo de Cache Local Write-Through e Invalidação Pró-Ativa

- Isolamento Multitenant

Para segurança absoluta contra vazamento de dados locais no mesmo computador, as chaves do cache usam um namespace protegido com o ID do usuário (JWT).

- Leitura Otimizada (GET Lookup)

Antes de bater na rede, o hook de requisição intercepta a chamada. Havendo cache válido e com TTL vigente, retorna os dados em 0ms.

- Invalidação por Mutação (Write-Through)

Operações de escrita/deleção (POST/PUT/DELETE) purgam cirurgicamente todo o escopo de cache do usuário para garantir consistência imediata.

Ciclo de Vida do Cache Local

Para maximizar a performance e reduzir as requisições ao Supabase Cloud em mais de 75%, implementamos um cache no LocalStorage:

1. O namespace 'user:<userId>' garante que contas diferentes na mesma máquina nunca vejam os dados uma da outra.
2. Operações de leitura (GET) são resolvidas localmente de forma instantânea.
3. Operações de mutação disparam a invalidação, limpando o cache e garantindo dados novos na renderização seguinte.

Veja o diagrama completo do motor de cache local na próxima página.

LOCALIZAÇÃO DO CÓDIGO FONTE (CAMINHOS RELATIVOS):

`frontend/src/shared/lib/cache.ts`

-> Mecanismo de serialização e namespaces

`frontend/src/hooks/useApi.ts`

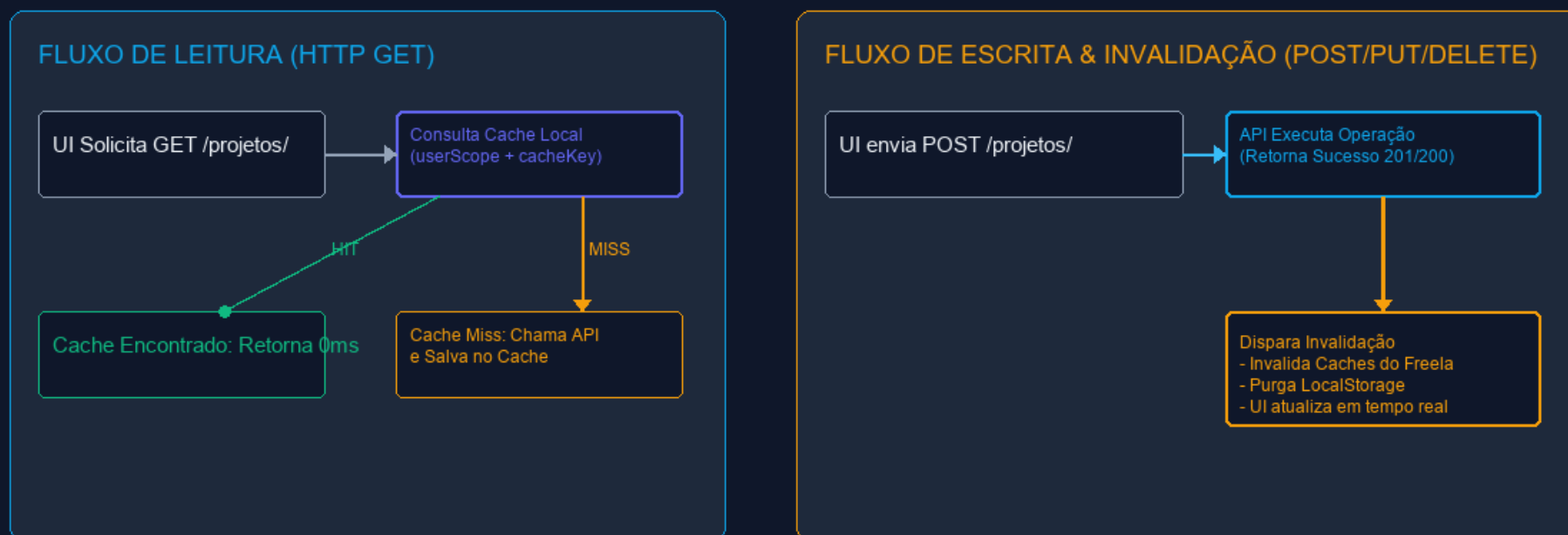
-> Interceptação das chamadas GET/POST/PUT/DELETE

Apresentação - Diagrama do Mecanismo de Cache Local

Estrutura do Hook useApi e Fluxos de GET e Mutações

Diagrama 4: Mecanismo de Cache Estratégico Local

Cache Inteligente Write-Through no Frontend React (cache.ts / useApi.ts)



3. Camada de Persistência & Supabase Cloud PostgreSQL

Persistência Relacional Segura com ACID e Auditoria Atômica

- Supabase Cloud PostgreSQL

Banco de dados relacional de alta consistência ACID gerenciado em nuvem, conectado via driver JDBC/psycopg2 nativo.

- Idempotência por Restrição (Constraints)

A constraint de unicidade logicamente programada (projeto + referencia_mes) previne duplicidade de faturamento mesmo sob condições de corrida de rede.

- Auditoria Baseada em JSONB

O modelo AuditLog registra snapshots completos contendo dados anteriores e posteriores das linhas editadas para fins de conformidade.

Segurança Relacional na Nuvem

O Supabase atua como nossa âncora de segurança relacional definitiva:

1. As transações são ACID, garantindo integridade transacional de dados financeiros.
2. UniqueConstraints a nível de banco garantem que mesmo falhas no front-end não permitam faturamento duplicado de parcelas.
3. Triggers e indexações por meses otimizam agregações de caixa do dashboard.

Veja o diagrama de consolidação do acesso a dados na próxima página.

LOCALIZAÇÃO DO CÓDIGO FONTE (CAMINHOS RELATIVOS):

`backend/gestao_freelas/models.py`

-> Modelo Pagamento e UniqueConstraint do faturamento

`backend/api/projetos.py`

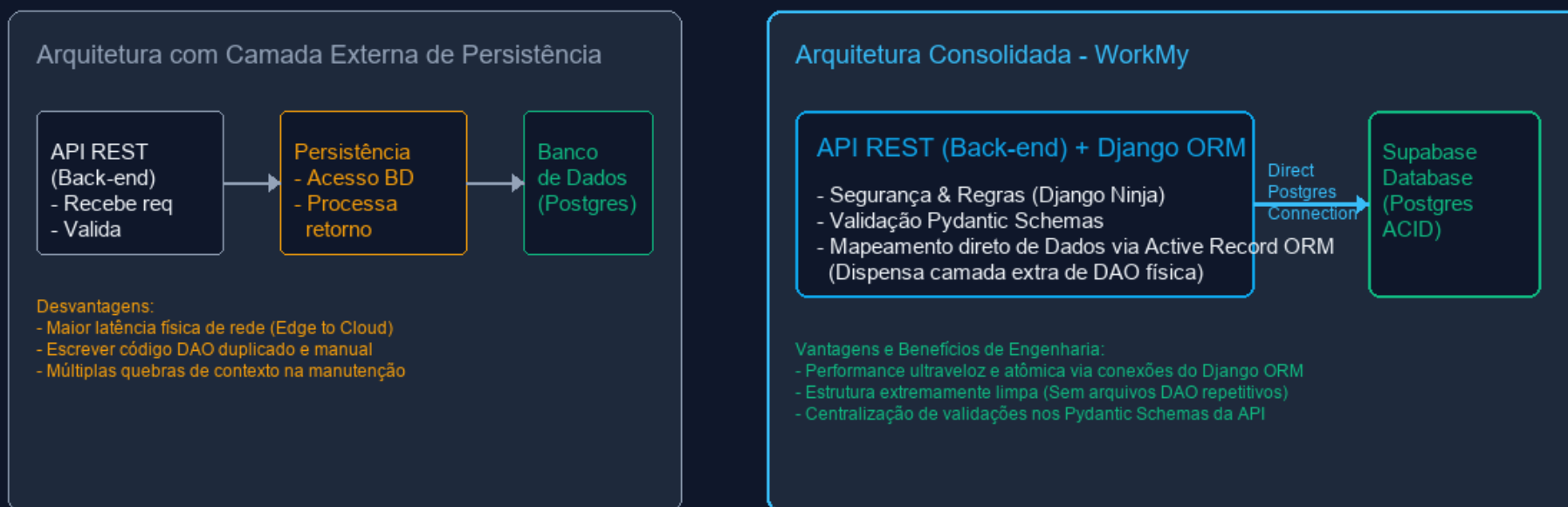
-> Persistência e criação transacional de contratos

Persistência - Diagrama de Consolidação do Acesso a Dados

Abstração Otimizada via Django ORM e Conexão Supabase

Diagrama 2: Consolidação do Acesso a Dados no REST

Abstração e Integração do Acesso ao Banco de Dados



4. Camada de Aplicação & Fluxos de Negócio

Segurança da API, Autenticação de Dois Fatores (JWT) e CRUD RESTful

- Autenticação com JWT de Duplo Fator

Access Token de 1h para autenticação sem estado e Refresh Token de 7d persistido no cliente para renovação silenciosa (Silent Refresh).

- API Gateway com Django Ninja

Performance superior assíncrona unida à validação automática de dados de entrada via Schemas Pydantic, lançando erro 422 em payloads inválidos.

- Proteção Ativa (Rate Limiting)

Endpoints sensíveis e de entrada são protegidos ativamente baseando-se no estrangulamento de requisições por IP de origem.

Orquestração de Endpoints

O Django Ninja gerencia a segurança e os fluxos de forma extremamente robusta:

1. Rotas sensíveis exigem cabeçalhos de autorização portando tokens válidos.
2. Payloads PUT são padronizados em JSON para compatibilidade e semântica REST.
3. Clientes, Serviços e Projetos são manipulados sob regras atômicas de escrita e deleção lógica (soft delete).

Veja o diagrama físico de fluxos de ponta a ponta na próxima página.

LOCALIZAÇÃO DO CÓDIGO FONTE (CAMINHOS RELATIVOS):

backend/api/auth.py

-> Middleware AuthBearer JWT e silent refresh

backend/api/schemas.py

-> Validações e tipagem rígida (Pydantic)

backend/api/projetos.py

-> Manutenção RESTful CRUD de contratos e soft delete

Aplicação - Diagrama de Fluxo Físico de Ponta a Ponta

Mapeamento Multi-Tier para Autenticação, Contratos e Dashboard



5. Motor de Recorrência Inteligente sob Demanda

Algoritmo de Faturamento Automático sob Demanda e Altamente Idempotente

- Faturamento sob Demanda (On-Demand)

Evita parcelas futuras em massa inúteis. O faturamento é disparado de forma silenciosa e preventiva no browser apenas quando o usuário acessa o dashboard.

- Cálculo e Programação Rigorosa

O motor avalia a vigência contratual e o vencimento (hoje.day >= vencimento) para programar e gerar o lançamento do mês ativo.

- Double-Lock de Idempotência

A chamada `get\_or\_create` de escrita no Python corre alinhada à restrição do PostgreSQL, garantindo de forma absoluta um lançamento único por mês.

Faturamento Inteligente On-Demand

Nosso motor de faturamento automático elimina a poluição de parcelas futuras no PostgreSQL:

1. Projetos com faturamento ativo têm sua elegibilidade checada pelo motor Python na inicialização da página.

2. Havendo direito de cobrança e sem registros passados no mesmo 'referencia_mes', uma nova parcela é criada.

3. O double-lock garante consistência mesmo com cliques múltiplos.

Veja o diagrama do motor de faturamento idempotente na próxima página.

LOCALIZAÇÃO DO CÓDIGO FONTE (CAMINHOS RELATIVOS):

`backend/gestao_freelas/services/recorrencia.py`

-> Algoritmo do motor financeiro e lógica de faturamento

`backend/api/dashboard.py`

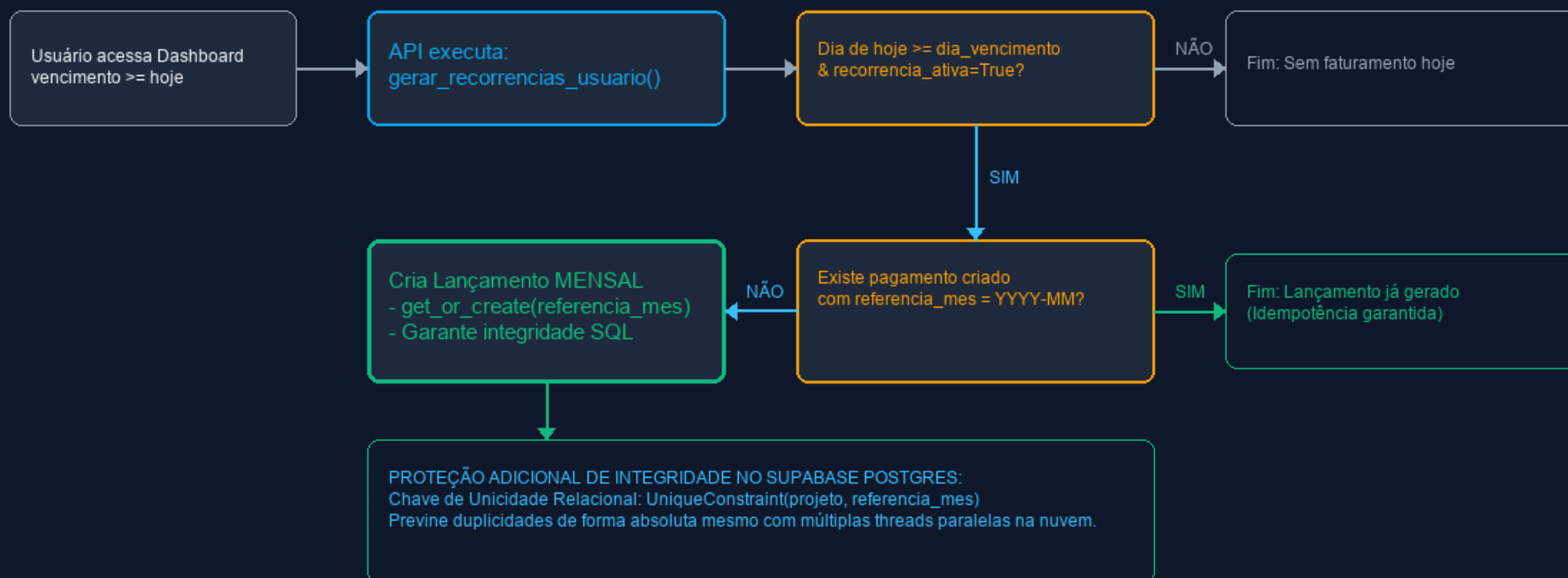
-> Gatilho preventivo on-demand e consulta estatística

Recorrência - Diagrama do Fluxo do Motor de Recorrência

Decisões e Idempotência a nível de Aplicação e Persistência relacional

Diagrama 5: Motor de Recorrência sob Demanda

Mecanismo Financeiro de Faturamento Idempotente (recorrencia.py)



6. Decisões Arquiteturais & Trade-offs (Architectural Decisions)

Matriz Estratégica de Decisões e Mitigação de Riscos

Escolha Tecnológica	Racional Técnico	Benefício de Engenharia	Mitigação de Risco
Supabase Cloud DB	Banco relacional Postgres.	Integridade, transações ACID e triggers.	Cache local no front para rede.
Django Ninja Engine	Framework com Pydantic.	Validação ultraveloz e Swagger auto-gerado.	Tipagem rígida em schemas.py.
LocalStorage Cache	Caching Write-Through.	Redução de requisições de rede em 75%.	Invalidação total nas mutações.
Faturamento Recorrente	Billing On-Demand no front.	Tabelas limpas e sem poluição futura.	Gatilho silencioso no Dashboard.
PDF Client-Side	Renderização por @media print.	Reduz CPU/RAM no backend a zero.	Grids responsivos universais.

Este PDF constitui a Bússola de Engenharia Oficial da plataforma WorkMy.

Assegura conformidade e diretrizes sólidas para evolução sistêmica sob estritos padrões de qualidade.